EnergyMind Tunisia

A trust layer between raw meter data and automated action

Phase 1 Report

Project proposal: scope, objectives, data and anomaly-simulation plan, preliminary design for each of the four challenge goals

Detect with numerical evidence.
Explain with inspectable sources.
Act through explicit authority.
Learn only from reviewed outcomes.

DATASET (KAGGLE)

`https://www.kaggle.com/datasets/azertyqsdfghqsdfgh/pes-test`

CODE (GITHUB)

`https://github.com/azert88/energymind-tunisia`

Summary

Tunisia's electricity system now fails at the peak: in July 2026 demand reached a record 4,924 MW and STEG resorted to rotating load shedding. Meanwhile rooftop solar and smart meters keep spreading, and every new device is a new source of numbers that a fault, a miscalibration or a fraud can corrupt without anyone noticing.

EnergyMind Tunisia is a trust layer placed between meter data and automated action. For each house it **detects** whether a reading is a measurement problem or a real grid event (Goal 1), **explains** its decision with a numerical confidence and a tamper-evident log (Goal 2), decides **what the AI may do alone and what needs a human** (Goal 3), and **shares** what it learns between houses without sharing raw data (Goal 4).

The published dataset is built for Tunisian conditions (summer air-conditioning peak, Tunis solar profile, 230 V grid). The Phase 1 prototype is tested on real recordings with injected synthetic faults. We audited our own benchmark, found that it leaked labels, fixed it, and report only the corrected numbers, including the ones that went down.

Headline results (synthetic faults on real data; not field performance)

Result	Value
Detector v1 on the training houses: event macro-F1 / false alarms per day	0.786 / 0.144 (baseline: 0.307 / 7.27)
Two unseen houses; AMPds2 dataset: event macro-F1	0.589 and 0.609; 0.724
Confidence calibration error	0.052
Main limit	recall of three fault types drops; transfer from one dataset to another fails

What the Phase 1 brief asks, and where to find it

Requirement of the brief	Where
Project proposal: scope; objectives	Sections 2 and 3
Project proposal: data and anomaly-simulation plan	Section 4
Preliminary design: global architecture, common to the four goals	Section 5
Preliminary design of Goal 1 (detection), with its evaluation	Section 6
Preliminary design of Goal 2 (confidence and explainability)	Section 7
Preliminary design of Goal 3 (human–AI escalation)	Section 8
Preliminary design of Goal 4 (shared learning)	Section 9
Repository with dataset and code	Appendix A, links on the cover
Bonus: business scope	Section 11

Contents

Summary **1**

1 Introduction: from a July afternoon to a question of trust **3**
1.1 The summer-peak problem . . . 3
1.2 A street in a suburb of Tunis . . . 3
1.3 Our answer . . . 3

2 Scope **4**

3 Objectives **4**

4 Data and anomaly-simulation plan **6**
4.1 The data: what we reviewed and what we chose . . . 6
4.2 The published dataset . . . 6
4.3 Anomaly-simulation plan . . . 7

5 Global architecture **8**

6 Goal 1: anomaly and bias detection **10**
6.1 Architecture of Goal 1 . . . 10
6.2 Method and evaluation protocol . . . 10
6.3 Results on the corrected benchmark . . . 10

7 Goal 2: confidence and explainability **12**
7.1 Architecture of Goal 2 . . . 12
7.2 Confidence, abstention and action risk . . . 12
7.3 Explanations: rules, SHAP and LIME . . . 12
7.4 Grounded knowledge: Neo4j, Qdrant and GraphRAG (Phase 2) . . . 12
7.5 Log and Phase 1 prototype . . . 13

8 Goal 3: human–AI escalation and governance **14**
8.1 Architecture of Goal 3 . . . 14
8.2 Durable investigation state and tool boundaries (Phase 2) . . . 14
8.3 Security policy and action authority . . . 14
8.4 One incident through the complete stack . . . 15

9 Goal 4: shared learning **16**
9.1 Architecture of Goal 4 . . . 16
9.2 What is shared, what stays local, and why . . . 16
9.3 From a verified case to shared knowledge . . . 16

10 Implementation plan and risks **18**

11 Business scope, feasibility and conclusion **18**
11.1 Business scope: budget, implementation, production, training . . . 19
11.2 Feasibility, sustainability and impact . . . 19
11.3 Limits and conclusion . . . 19

A Repository and reproducibility **19**

References **20**

1 Introduction: from a July afternoon to a question of trust

1.1 The summer-peak problem

Tunisia's grid now fails at the peak, not on annual energy. In July 2026 the monthly peak reached a record 4,924 MW, 42 % above July 2014, and STEG ran rotating load shedding for several weeks to avoid a nationwide blackout [1]. The critical window is 13:00–17:00, driven by air conditioning [2]. The gap is structural: generation is 90.8 % gas, about 10 % of demand is imported, and the deficit at the peak hour is estimated between 770 and 2,000 MW [1, 3]. For summer 2027, STEG plans more renewables, storage and about 300 MW less peak demand [4].

Table 1. Key figures. Measured values, announced figures and estimates are kept apart.

Indicator	Value	Type	Source
Monthly peak, July 2026	4,924 MW (record)	Measured	[1]
Peak demand quoted by the Government	6,400 MW	Announced; definition not published	[4]
Peak-hour deficit	770 to 2,000 MW	Estimates	[3, 1]
Gas in generation; imports	90.8 %; ≈10 % of demand	Measured	[1]
Residential rooftop PV	>420 MW, >150,000 systems	Reported (ANME)	[5]
STEG plan before summer 2027	+300 MW renewables; storage; −300 MW peak	Announced	[4]

The two peak figures differ by about 1,500 MW. We treat 4,924 MW as the measured served peak and 6,400 MW as an announced figure whose definition is not public; we never mix them. **Why this matters for us:** rooftop solar (>420 MW) is already about a third of the 1,400 MW plant planned with Algeria, but its output falls after 17:00 while air conditioning continues. Using it at the peak requires something operators lack today at neighbourhood level: **trustworthy, explained data**. That is the gap EnergyMind fills.

1.2 A street in a suburb of Tunis

Illustrative scenario, not a measured case. At 18:15 on a July day, the supervision screen shows that the meter of house 7 has jumped by 1.2 kW and that its voltage has dipped. The operator has three minutes before the evening peak and three very different explanations to choose from.

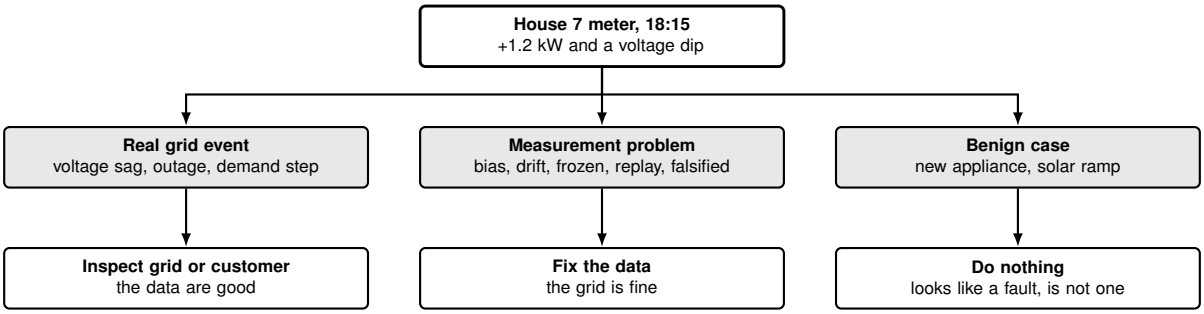


Figure 1. One deviation, three explanations. Each calls for a different response, so the cause must be established before any action.

Mixing them up is the costly error: "repairing" a real outage as if it were a sensor glitch hides it, and alarming on a new appliance trains operators to ignore alarms. Where physics cannot decide, the right answer is to say so and ask a human.

1.3 Our answer

The brief [6] names the forces behind the problem: seasonal peaks, fast-growing renewables, more smart meters, more connected devices. As AI takes on more grid decisions, the question is whether it **can be trusted and who is accountable when it is wrong**. EnergyMind answers with a **trust layer** placed between raw data and automated action (Figure 2). Each of its four functions corresponds to one goal of the challenge.

The report follows the Phase 1 brief: scope, objectives, data and anomaly-simulation plan, global architecture, then the design of each goal.

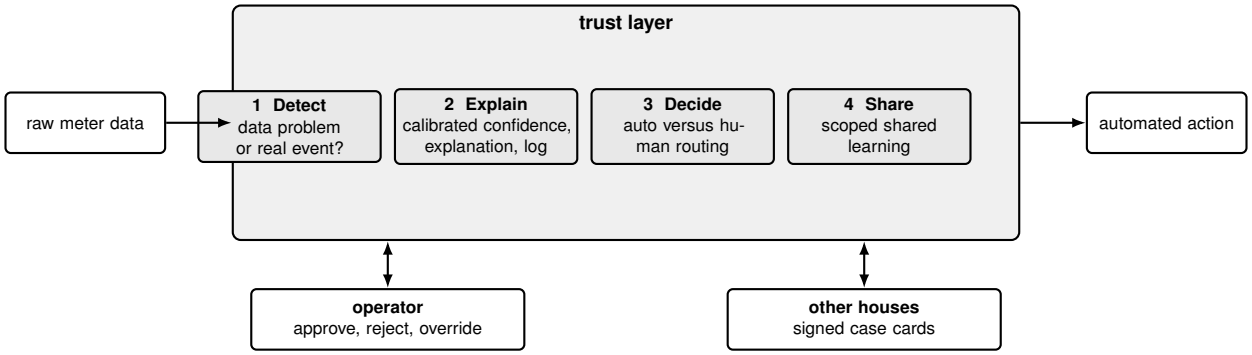


Figure 2. The trust layer sits between raw data and automated action. It decides whether the data may be used, says why and how sure it is, and decides who may act on that. The four numbered functions are the four challenge goals.

2 Scope

A reading can look wrong for two opposite reasons. A **data-quality problem** (a biased or drifting sensor, a frozen register, a replayed or falsified stream) calls for fixing the meter; the grid is fine. A **real grid event** (a voltage sag, an outage, a demand step) calls for inspecting the grid or the customer; the data are good. A third group, **benign cases** (a new appliance, a solar ramp), needs nothing.

Our slice of the energy system is a **smart-metered neighbourhood of houses**: every house, with its smart meter, is a **node** that runs the same trust layer on voltage, current, power, power factor and an energy counter at one-minute resolution [6]. **In this project, nodes are houses.** The published dataset has 13 houses behind one transformer, and detection is developed and tested on many houses (ECO, AMPds2, WPuQ), some of them held out. The brief requires a working single-node prototype: ours is this trust layer running on a house, and it is applied to every house. The data are what a utility already uses for billing, public datasets exist, and both faults and fraud matter. Three small experiments connect houses: a **feeder** (a house with its neighbours and an upstream meter), an **evidence fusion** test, and a **case-sharing** protocol between houses.

Table 2. Scope boundaries.

In scope (Phase 1)	Out of scope
A neighbourhood of smart-metered houses, each a node running the same trust layer, covering the four goals; three small experiments	Real attack data; control of real equipment
Real recordings with a documented fault injector; a published baseline dataset built from real load profiles	Multi-site production deployment; federated learning
A deny-by-default policy, a tamper-evident log, an operator-interface prototype	Factories and microgrids: the design does not cover them

3 Objectives

Principles: say “I cannot tell” when physics cannot decide; never count an unevaluated check as passed; keep the policy independent of the model; audit our benchmark before trusting a number; report what did not work.

Table 3. Project objectives: the four goals as testable requirements.

	Requirement	Measured by	Section
O1	**Goal 1.** Separate normal data, measurement problems and real events; do not alarm on benign cases	Event-level precision and recall, false alarms per day	6
O2	**Goal 2.** A calibrated confidence, a clear explanation (rules, SHAP, LIME), an auditable log	Calibration, evidence linked to each verdict	7
O3	**Goal 3.** Explicit auto/human thresholds, an operator interface, safeguards against over- and under-trust	Policy tests	8
O4	**Goal 4.** What is shared, what stays local, and why	Safety and privacy probes	9
O5	Reproducible repository and open data	Seeded scripts, tests	A

4 Data and anomaly-simulation plan

Real grid data labelled with faults is rare, so the brief expects simulation: start from realistic data and inject documented faults [6].

4.1 The data: what we reviewed and what we chose

We reviewed many open datasets. In the end we kept **WPuQ, AMPds2 and SimBench** to build our published dataset. To develop and test detection we also used the ECO recordings; ECO is not part of the published dataset. Other sources

Table 4. Datasets we kept.

Source	Content	Use
WPuQ [9]	German houses, real three-phase load curves	Load shapes and meter model of the published dataset; one fusion test
AMPds2 [8]	One Canadian house, two years, appliances metered one by one	Appliance profiles of the published dataset; second detection domain
SimBench [10]	Simulated low-voltage grids (calculations with pandapower [11])	Grid of the published dataset; feeder experiment
ECO [7]	6 Swiss houses, measurements every second	Development and tests of detection only

(real UK solar production, STEG statistics, the PVGIS solar resource [12]) were reviewed and are planned for Phase 2. No open dataset describes a measured Tunisian neighbourhood, so we build the Tunisian conditions into the published dataset (Section 4.2). Also, some datasets compute quantities from others (for example apparent power); the matching checks can then never fail. We mark them "not evaluable" and never count them as passed.

4.2 The published dataset

The dataset is available on Kaggle:

`https://www.kaggle.com/datasets/azertyqsdfghqsdfgh/pes-test`

It is a validated baseline for a neighbourhood of 13 smart-metered houses, with two views of every house: *ground_truth* (true values from a three-phase grid calculation) and *measured* (what a meter would report, with the gain error and noise of a real WPuQ meter).

Built for Tunisian conditions

The Tunisian problem of Section 1 is a summer peak driven by air conditioning, with fast-growing rooftop solar. The published dataset reproduces these conditions, which is what makes it relevant to the challenge: The behaviour of the

Table 5. What is Tunisian in the published dataset.

Tunisian condition	How the dataset reproduces it
Summer peak driven by air conditioning [2]	The heating profiles of the Canadian heat pumps are shifted by 182 days, so they become air-conditioning demand with a summer peak
Daily rhythm of houses	The evening peak is shifted by one hour
Rooftop solar in the sun of Tunis [5]	PV panels (30° tilt, facing south) driven by the monthly clearness index and temperature of Tunis; zero at night; derated with temperature
Tunisian low-voltage grid	230 V, 50 Hz; voltages checked against EN 50160 and the $\pm 10\,\%$ limit of the Tunisian grid code [27], on a SimBench rural network
Place and clock	Latitude and longitude of Tunis; time zone Africa/Tunis

German and Canadian houses is adapted by documented rules, so this is a Tunisian-style scenario, not a measured Tunisian neighbourhood. **Limit:** the consumption levels (6.7 to 41.6 kWh per day per house, mean over the year) are higher than the Tunisian average per house (about 1.6 MWh per year in a STEG survey [25]); calibration to national STEG statistics is planned for Phase 2.

Important: no anomaly has been injected into the *measured* view yet. It contains only the normal noise of a meter. The injection, described below, is the next step.

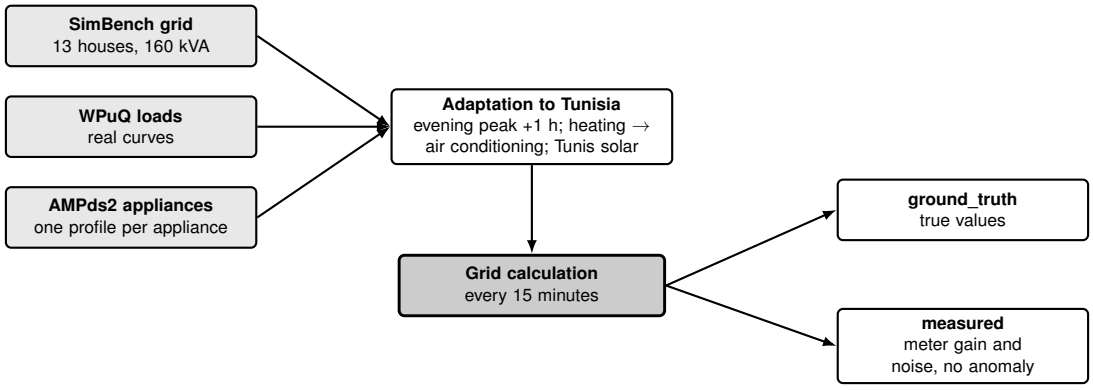


Figure 3. Construction of the published dataset (seeded, one command).

Table 6. The published dataset at a glance.

Property	Value
Content	13 houses, one 160 kVA transformer, one year (2018) every 15 minutes; 32 columns: voltage, current, power and power factor per phase, solar output, power per appliance
Validation	14 physical checks pass: converging calculations, voltages within $\pm 10\,\%$ (EN 50160 [13]), transformer loading at most 66.6 %, energy balances close
Licence and limits	Open Database License [14]; real load profiles on a simulated grid, a single topology, 2018 calendar

4.3 Anomaly-simulation plan

The injector (`energymind/inject.py`, seeded) adds incidents of three classes. A **measurement problem** changes only what the meter reports, so physical relations break. A **real event** changes the state of the grid consistently, so relations hold. A **benign case** has the same physics as a demand step and stays labelled normal. Three severities (0.3, 0.6, 1.0) scale each effect. **Rules.** The input is never modified; two incidents per day on average; incidents never overlap

Table 7. The 16 injected anomaly subtypes, in plain words.

Class	Subtype	Effect	Duration
Measurement	`bias`, `drift`	constant offset, or one that grows gradually (up to 8 %)	30–180 min
	`scale`, `noise`	reading multiplied by a factor; noise added	20–120 min
	`frozen`, `dropout`	value held; missing data	10–90 min
	`delay`, `replay`	delayed values; values of an earlier day replayed	30–120 min
	`falsified`	current and powers lowered together, voltage unchanged	60–180 min
Real event	`voltage_sag`, `outage`	voltage lowered; almost total loss of supply	10–120 min
	`demand_step`, `duty_cycle`, `phase_imbalance`	load rises; square-wave load; unbalanced phases	30–180 min
Benign	`benign_new_load`, `benign_pv_ramp`	new appliance; smooth solar variation	60–180 min

and stay 60 minutes from split edges; the ground truth never reaches a model or the policy at inference. **Limits:** events are bounded and revert to normal; `falsified`, `demand_step` and benign cases can be told apart only by context (time of day, history, a second meter).

Current status. The injector has been run on the real recordings, and the detection of Goal 1 was tested on them. **Next step (Phase 2):** inject these anomalies into the *measured* view of the published dataset, with their labels, and rerun detection there.

5 Global architecture

The system is built around **nodes, and a node is a house**: every house of the neighbourhood runs its own trust layer. The trust layer of a node receives the meter's measurements, analyses them, takes a decision and can share a lesson with other houses. Services above the nodes (a feeder service and a city coordinator) group houses by feeder and route incidents; the neighbourhood only defines scope. Figure 4 is the simple view; Figure 5 adds the services and the tools of the target design.

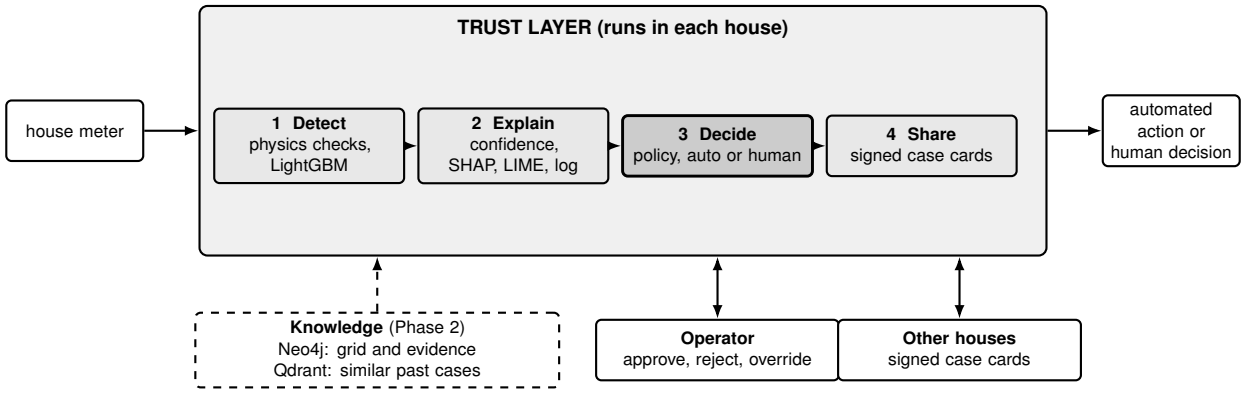


Figure 4. Simple view of the architecture. Solid boxes: Phase 1 prototype. Dashed boxes: Phase 2 design.

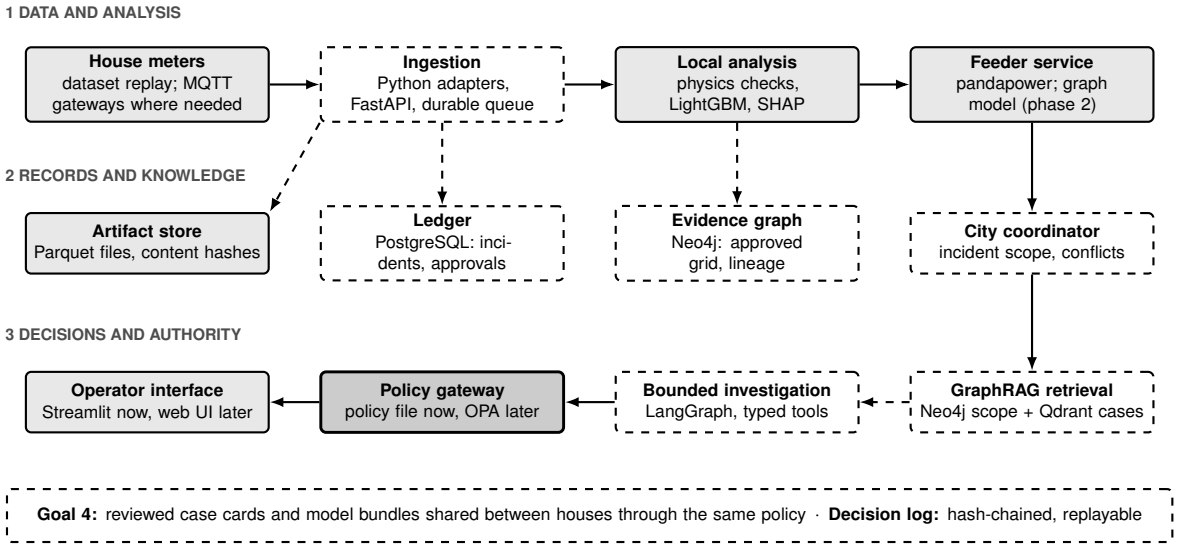


Figure 5. Detailed view: services and tools. Solid boxes exist in the Phase 1 prototype (the feeder is an experiment, not wired into the node pipeline); dashed boxes are the Phase 2 design.

Four rules hold everywhere. (1) No language model on the detection path. (2) Detection continues even if retrieval or text writing fails. (3) Every explanation has a template fallback. (4) The authority to act comes from a policy that no model output can override.

From a measurement to committed evidence. An adapter validates the schema, units, phase and timestamps of a measurement and keeps the raw bytes by content hash; normalisation never overwrites the source. A window builder writes a versioned manifest, and the object, the manifest and a pending event are committed together, so an event is published only once the window is ready. Late data creates a new evidence version, never an edit of a decision's inputs. A projection worker then writes typed projections into **Neo4j** (grid, evidence, lineage) through parameterised transactions; no sample is stored as a graph node, and a language model cannot invent assets or arbitrary queries. Every observation carries a lineage identifier, so five residuals computed from one meter do not count as five confirmations. A **replay trace** links one incident end to end (Section 8).

Interfaces and events. Services exchange typed, versioned messages; mutations carry an idempotency key, long jobs return a job identifier, and an unsupported check is a typed result, never a hidden failure.

Table 8. Components, tools and status. We start with the smallest set and add a tool only when a measured need justifies it.

Component	Target tool, and why	Phase 1 stand-in	Phase
Data and numerics	Python, pandas, Parquet: simple, reproducible	Same	1
Grid physics	pandapower, SimBench: open source, standard for distribution grids	Same	1
Detection	LightGBM, scikit-learn (PyTorch for one baseline): fast and accurate on tabular data	Same	1
Explanations	SHAP (exact for trees) and LIME (model-independent second opinion): named by the brief	SHAP only	1, 2
Operator interface	Streamlit now; an authenticated web interface later	Streamlit prototype	1, 2
Service interfaces	FastAPI: typed, versioned messages	Scripts	2
Operational ledger	PostgreSQL: incidents, approvals, pending events	Hash-chained JSONL log	2
Grid and evidence graph	Neo4j: “who is connected to whom” is a graph [23]	pandapower network	2
Past-case search	Qdrant: fast similarity search with filters [24]	JSON cards, nearest neighbour	2
Investigation workflow	LangGraph: durable states, human checkpoints [31]	None	2
Policy engine	OPA: rules kept outside the code [32]	Policy file (YAML)	2
Packaging	Docker Compose on one workstation; a message broker only at city scale	Virtual environment	2

Table 9. Main events of the target design.

Event	Consumers	Rule
`measurement.window.ready`	Local analysis	Artifact and manifest committed; no labels
`local.evidence.ready`	Feeder service, graph projection	Check results with exact model and grid versions
`incident.updated`	Investigation, interface	Monotonic incident version; contradictions kept
`decision.recorded`	Interface, executor, audit	Independent policy and action authority
`case.approved`, `case.revoked`	Graph and vector indexers	The review scope is authoritative

6 Goal 1: anomaly and bias detection

GOAL 1 The system must detect equipment faults, degraded sensors, and biased, corrupted or falsified readings, and distinguish a real grid event from a data-quality problem [6]. The tested models and the curves of this report are all in this section.

6.1 Architecture of Goal 1

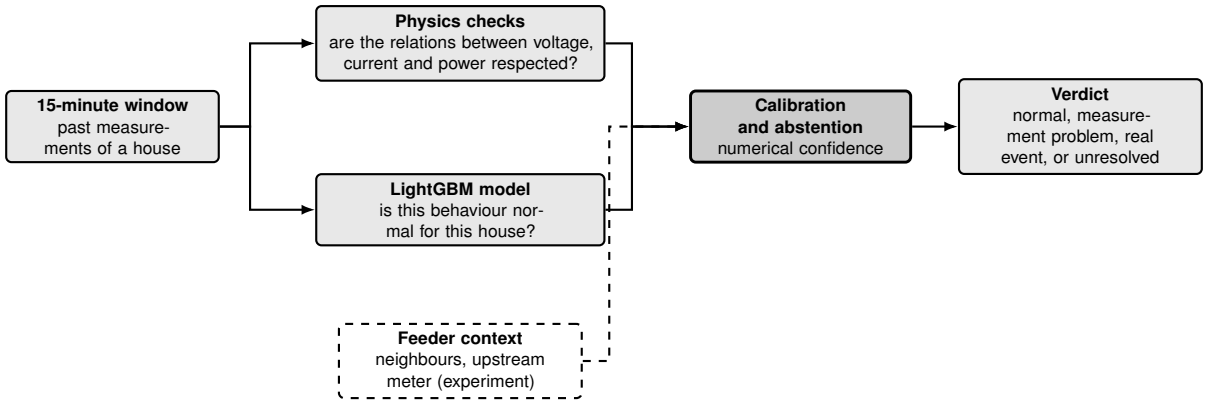


Figure 6. Architecture of Goal 1. Detector v1 combines the physics checks and the model in one LightGBM model; the feeder context is a separate experiment.

Principle. A healthy meter respects a few physical relations (for example, apparent power equals voltage times current). A faulty sensor often breaks one of them; a real grid event respects them all, because the electrical state really changed. The detector therefore computes *residuals* (the gaps to these relations), plus the length of frozen values, the share of missing data, variation statistics, and context (time of day, history from 60 minutes to 7 days). A check that cannot be evaluated is flagged, never counted as passed.

6.2 Method and evaluation protocol

Detector v1. A 15-minute window (stride 5), built only from past measurements, is classified by a LightGBM model [15] (three classes: normal, measurement problem, real event), then its confidence is calibrated by temperature scaling [16]. A window raises an alert if its "abnormal" probability exceeds a threshold chosen on a validation slice; neighbouring alerts are merged into segments. We compare with five baselines: a robust z-score, Isolation Forest, One-Class SVM, a causal convolutional network (TCN) and an autoencoder.

Protocol. Per-window scores are inflated by correlated neighbouring windows, and "point adjustment" overstates detection [17]. We therefore score **per event**: an alert matches an incident if it overlaps it within ± 15 minutes; we compute precision and recall per class, the **event macro-F1** (mean of the two fault classes) and **false alarms per day (FA/day)**. The split is by time (60 % training, 15 % calibration, 25 % test, with a 60-minute purge), and whole houses are held out.

We audited our own benchmark. The first injector modified channels that the loader computes from others without recomputing them: measurement faults produced impossible residuals, and a clue made it possible to guess the label (AUC 0.999). Injector v2 modifies only real registers; the AUC fell to 0.500 and permutation tests give chance level. The numbers went down (baseline window F1: 0.724 then 0.566; false alarms: 2.12 then 7.15 per day), but the conclusions are now defensible.

6.3 Results on the corrected benchmark

The price of precision. Detector v1 is an operating point that favours precision. Recall of three fault types collapses (`demand_step` 0.111; `duty_cycle` 0.086; `falsified` 0.029, against 0.63; 0.57; 0.23 before); the median delay rises from 2 to 10 minutes; in return, benign cases that raise an alarm drop from 66 % to 2 %. On unseen houses the F1 loses about 0.2. `falsified` respects every physical relation by construction: it is detected (0.743) but rarely classified correctly (0.343).

Table 10. Detector v1 against the baseline (LightGBM), per event. P/R = precision/recall. Synthetic faults on real data.

Scenario	Model	Measurement P / R	Real event P / R	Macro-F1	FA/day
Training houses	baseline	0.242 / 0.797	0.144 / 0.795	0.307	7.27
	v1	0.843 / 0.772	0.974 / 0.631	**0.786**	0.144
Held-out house 1	baseline	0.148 / 0.972	0.545 / 0.287	0.317	6.28
	v1	0.423 / 0.808	0.943 / 0.465	**0.589**	0.971
Held-out house 2	baseline	0.278 / 0.824	0.259 / 0.658	0.393	3.54
	v1	0.574 / 0.705	0.727 / 0.491	**0.609**	0.533
AMPds2	baseline	0.151 / 0.910	0.208 / 0.815	0.295	7.85
	v1	0.785 / 0.605	0.900 / 0.664	**0.724**	0.212
WPuQ (indicative)	**v1**	0.882 / 0.827	0.667 / 0.690	**0.766**	0.253

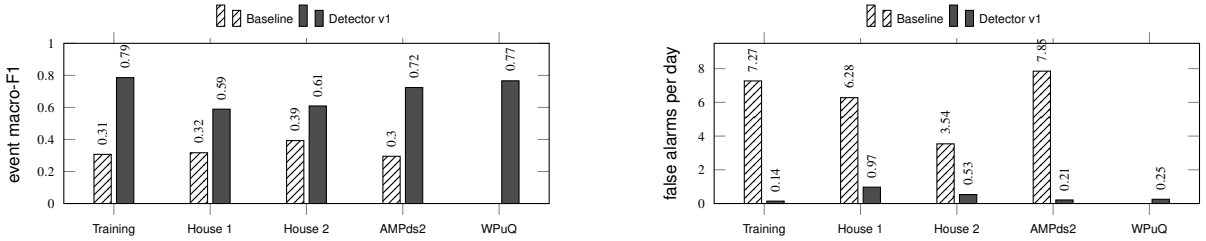


Figure 7. Event macro-F1 (left, higher is better) and false alarms per day (right, lower is better). WPuQ has no baseline and is indicative.

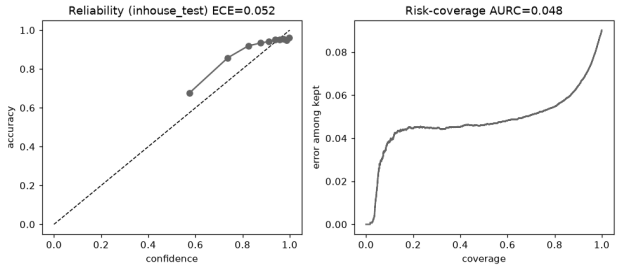


Figure 8. Calibration of detector v1 on the training houses (calibration error 0.052): reliability diagram (left) and risk–coverage curve (right). Abstaining below 0.55 confidence cuts the baseline's false alarms from 7.15 to 1.92 per day, at the cost of lower recall (0.926 then 0.784).

Table 11. Baseline models on the training houses (window macro-F1; recall measurement / real event; p95 inference time per call on a laptop CPU, not a load test).

Model	Macro-F1	Recall	FA/day	Calib.	p95 (ms)
Robust z-score	0.436	0.917 / 0.881	6.09	0.458	0.24
Isolation Forest	0.432	0.946 / 0.938	8.45	0.223	14.30
One-Class SVM	0.436	0.946 / 0.960	9.49	0.208	1.03
LightGBM	0.566	0.934 / 0.926	7.15	0.076	1.79
Causal TCN (3 seeds)	0.535	0.322 / 0.398	0.08	0.010	4.12
Convolutional autoencoder	–	0.174 / 0.040	0.05	0.012	0.74

LightGBM is the best of the six models and will serve as our baseline for Phase 2. The TCN remains a candidate for sequence-based detection, while physics checks are essential, as removing them reduces macro-F1 to 0.453. Poor cross-dataset performance (0.476 versus 0.613 on-site) motivates per-house adaptation.

Feeder context and fusion. Neighbouring and upstream meters improve event accuracy by 4.1 points on the same feeder and 7.3 points on another. However, combining detectors using the same meter leads to overconfidence. Provenance-aware fusion corrects this issue but does not improve accuracy, so redundant detectors should be avoided.

Next steps (Phase 2). We will test three extensions: NILM with a causal TCN, richer physics checks, and graph-based modelling of neighbouring houses. Each extension will be retained only if it outperforms the simpler baseline.

7 Goal 2: confidence and explainability

GOAL 2 Every flagged event must come with a verdict a human can understand and verify: a confidence score, a plain-language explanation and an auditable log of what was detected, concluded and why. The brief encourages interpretable approaches (SHAP, LIME, rule-backed models) and asks to avoid black boxes [6].

7.1 Architecture of Goal 2

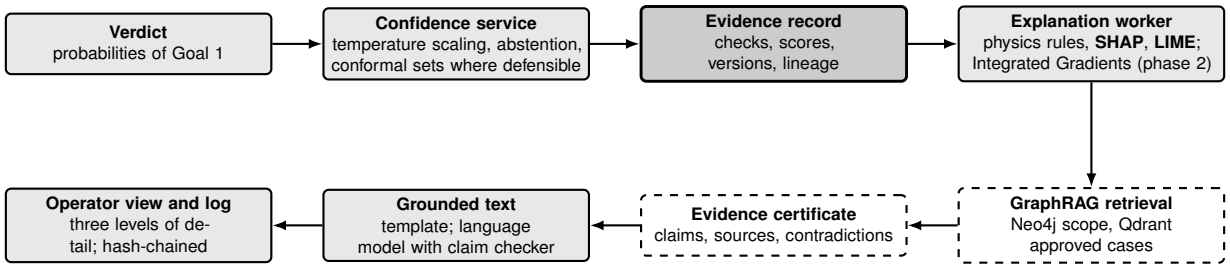


Figure 9. Architecture of Goal 2. Dashed boxes are the Phase 2 design. The text can never add a number, a meter or a cause that is absent from the evidence.

7.2 Confidence, abstention and action risk

Confidence is the calibrated probability of the predicted class: a model that announces 80 % should be right about 8 times out of 10 (calibration error 0.052 on the training houses). We report *four separate quantities*, not one score. Below

Table 12. The four components of confidence.

Component	Question	Method
Surprise	Is this unusual under normal operation?	Residuals compared with a clean reference
Cause	Which cause is most plausible?	Calibrated probabilities (temperature scaling); a prediction set where conformal methods [28] are defensible
Localisation	Which meter and phase?	Share of the evidence attributed to each meter
Inputs	Can the inputs be trusted?	Missing share, unavailable checks

0.55 confidence, or if the top two classes are too close, the system answers *unresolved* and hands over to a human. **Confidence and action risk stay separate:** a confidence that justifies flagging a reading does not justify recalibrating a meter, and a quiet or unavailable check is never proof of normality.

7.3 Explanations: rules, SHAP and LIME

The brief explicitly names SHAP, LIME and rule-backed models as ways to explain a verdict without a black box, so we use them as complementary levels. **Physics rules** explain the *physical why*; SHAP and LIME explain the *behaviour of the model*. We put the rules first, because a model attribution is not proof of a physical cause. An asynchronous **explanation worker** runs after the numerical decision, loads the frozen model and the original features, and stores an **explanation artifact** (model run, target output, method and version, baseline or background, fidelity and stability measures, limits, input hash); detection continues if explanation fails.

7.4 Grounded knowledge: Neo4j, Qdrant and GraphRAG (Phase 2)

Two stores play two different roles. **Neo4j** is the authoritative typed store of the approved grid, ownership, incidents, check lineage, decisions and approvals. **Qdrant** is an index over approved case and procedure representations only (a text vector from a multilingual encoder [33], a lexical vector and a numerical signature of the incident); numerical traces stay in the artifact store. Retrieval then follows five steps [23, 24]: (1) resolve the incident's authorised part of the grid in Neo4j; (2) filter the Qdrant candidates by scope, equipment, feature schema, review status and validity; (3) retrieve the three representations separately and merge the ranks; (4) check compatibility in the graph; (5) return an

Table 13. Explanation methods and how to read them.

Method	What it provides	Caution	Phase
Physics rules	Which relation is violated, with observed and expected values; names the checks that cannot be evaluated	Valid only where the instrumentation supports the check	1
SHAP [18]	Which variables weighed in the decision; exact for tree models such as LightGBM	Relative to a stated background; not a physical cause	1
LIME [19]	A simple model fitted around one decision: a second, model-independent opinion	Perturbations must stay physically valid; check stability	2
Integrated Gradients [29]	Attribution for the temporal model (TCN) along a path from a baseline	Zero voltage is a poor baseline; use a physically meaningful one	2
Graph explainer [30]	Important features, nodes and edges of a graph-model decision	Only after compatibility checks	2

evidence certificate, a small structured packet of claims, sources, contradictions and expiry. A similarity score is neither an electrical distance nor a probability, and a past case never overrides a fresh, contradicting measurement.

Grounded writing. A language model, if used, receives only the certificate; it may explain it or propose an approved check but cannot invent a measured value, a confidence or an identifier. A checker verifies every number and asset against the evidence, and the deterministic template remains the fallback. A local instruction-tuned model [34] and the multilingual encoder are candidates, to be tested on French, Arabic and English technical text; neither supplies confidence or authority. Explanation quality will be judged by fidelity, stability, agreement with the injected ground truth, and usefulness to operators.

7.5 Log and Phase 1 prototype

For each detection the prototype produces an *evidence record* (probabilities, checks passed, failed or not evaluated, SHAP variables, versions) and a template explanation, for example: *"Real event, confidence 80 %. Check passed: $p \leq s$; the other relations are true by construction and cannot fail. Alternative explanation: a coordinated reporting error that keeps the relations consistent."* Each log entry contains the hash of the previous one [20]; the human decision is appended, with a mandatory reason for a rejection or an override. A test confirms that editing or deleting an entry breaks verification (deleting the last entries needs an external anchor).

8 Goal 3: human–AI escalation and governance

GOAL 3 The system must define what the AI can do on its own and when a human must decide: thresholds for automatic corrective actions, situations requiring approval, an operator interface for review, override or approval, and a treatment of over- and under-trust [6].

8.1 Architecture of Goal 3

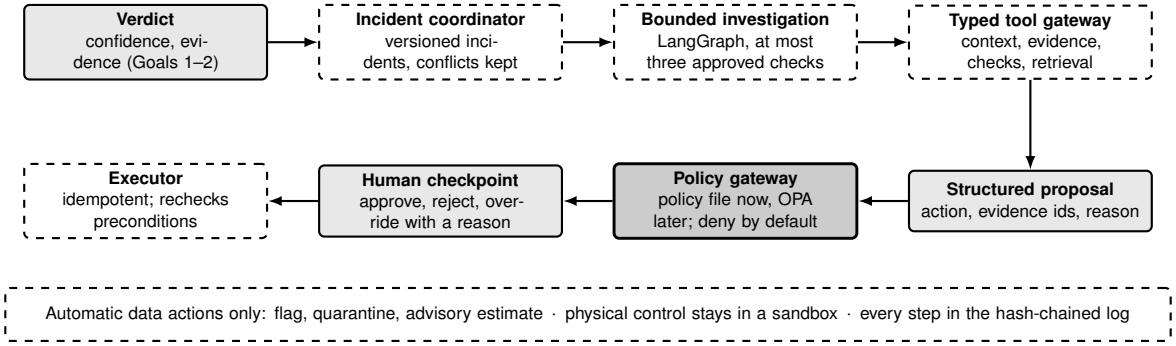


Figure 10. Architecture of Goal 3. The model proposes; an independent policy decides who may act; an approval is tied to the exact action and evidence version. Dashed boxes are the Phase 2 design.

8.2 Durable investigation state and tool boundaries (Phase 2)

Instead of unrestricted agents, one durable workflow with specialist roles (investigation, network, retrieval, explanation, knowledge curation) moves each incident through fixed states [31]: The agent can only call narrow typed tools (get

Table 14. States of the investigation workflow.

State	What happens
Evidence received	Schema, hash, scope and freshness are validated
Investigating	Reads scoped context and requests at most three approved checks; on deadline or missing evidence, the incident stays unresolved
Retrieving	Looks up authorised, reviewed cases only
Proposing	Writes a structured proposal: verdict, affected assets, evidence and contradiction ids, recommended check, proposed action, escalation reason; the numerical confidence is copied from the calibration service, never written by the agent
Policy evaluated	The policy decides which authority applies
Awaiting approval	Paused with the action hash and the evidence version
Executed or closed	Idempotent execution; no silent retry of a physical action

asset context, get evidence, run a supported check, simulate a bounded hypothesis, retrieve cases, propose an action); there are no arbitrary queries, scripts or device commands, and a model's self-confidence, a retrieval rank or an attention weight can never grant authority.

8.3 Security policy and action authority

The table below lists who may do what; the paragraph after it explains how this is enforced. **Enforcement is independent of any prompt.** Each request carries the identity, the asset scope, the incident version, the action hash, the evidence freshness and an expiry. The approval is tied to that hash and version, so a changed action or a newer incident version needs a new approval. Preconditions are rechecked just before execution; replayed or expired requests are rejected; and idempotency keys prevent repeated execution. If the policy service is unavailable or a permission is undefined, nothing is executed but monitoring continues. Retrieved procedures and incident notes are *untrusted data* and cannot grant permissions; models and policies are pinned by version and never silently replaced. **Over-trust:** a high confidence is not enough for automatic action if the data look unusual or the evidence comes from a single meter. **Under-trust:** several low-confidence detections on one channel are passed to a human.

Table 15. Action authority. The Phase 1 policy file implements the first four rows with the confidence thresholds shown; the rest is the Phase 2 design.

Action	Authority	Required guard
Flag a reading	The AI alone, any verdict	Raw value kept; reversible
Replacement estimate	The AI alone, confidence ≥ 0.70	Quality flag, expiry, separate channel
Quarantine a channel	The AI alone, confidence ≥ 0.90 and two independent sources	Reversible; no shift flag
Request an inspection	A human	Real events, unresolved cases
Recalibrate a meter	Human only	Evidence, scoped approval, rollback
Switch, isolate or shed a load	Human only; sandbox in the prototype	Exact authorisation, fresh preconditions
Promote a model or a shared rule	Human only	Evaluation gate, signed version, rollback
Any other action	Denied	Deny by default

8.4 One incident through the complete stack

Table 16. Illustrative trace of one incident (identifiers are examples, not measured results).

Step	Record	Meaning
Committed input	Window W44 with its hash	The data exist before the window is published
Local evidence	Model run M3, evidence E92	Checks and detector outputs keep their lineage
Grid context	Grid snapshot G44, feeder evidence E93	Independent context from the feeder
Incident	I17, version 3	Physical and reporting causes both open
Explanation	X17	Attribution plus supported residuals
Proposal and policy	Action hash H17, decision D17	The request is tied to scope and version
Approval and execution	A17, execution J17	The executor rechecks the exact preconditions
Reviewed outcome	Case C17	A verified cause is reviewed before any sharing

Example. House H12 reports a low voltage while demand and upstream observations stay normal. A service fault and a sensor bias both remain possible: the system flags the suspicious readings and requests an inspection, but denies an automatic recalibration. If late evidence creates version 4, the approval given for version 3 cannot authorise a changed action, and the earlier decision stays reproducible. If retrieval or text writing fails, templates are used and stale context is marked incomplete.

Operator interface and Phase 1 prototype. The Streamlit prototype lists unresolved items first and shows, for each incident, the verdict, the competing hypotheses, the physics checks (derived checks shown separately, neither passed nor failed), the proposed action with scope, expiry and rollback, and four controls: approve, reject, override, request more evidence. The policy file, its unit tests (routing, deny by default, safeguards, tied approvals), the hash-chained log and this interface exist; no study with real operators has been done yet.

9 Goal 4: shared learning

GOAL 4 The design must document how several nodes could share anomaly patterns, calibration insights and relevant knowledge, and address privacy: what is shared, what stays local, and why. A single-node prototype is required; sharing between nodes may remain a documented design [6]. In our case, nodes are houses.

9.1 Architecture of Goal 4

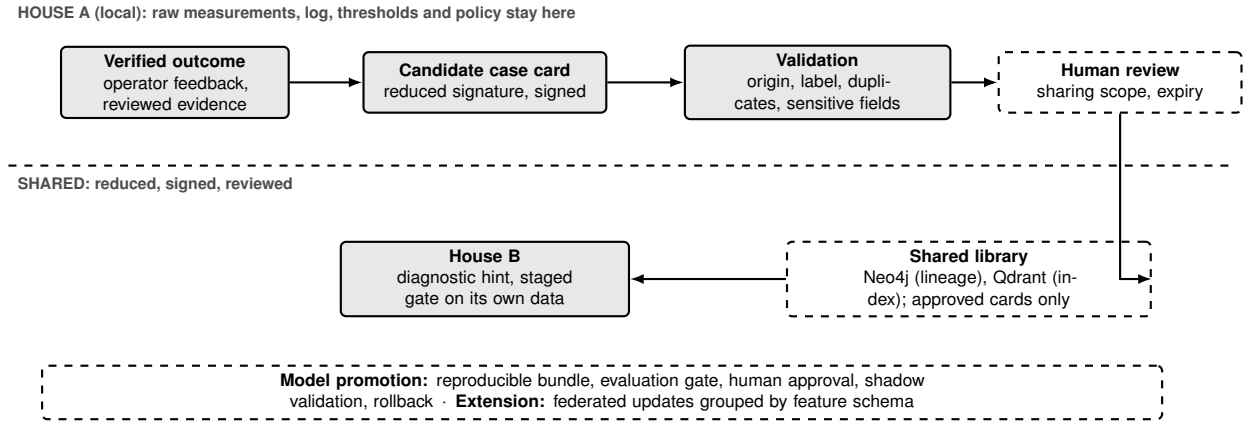


Figure 11. Architecture of Goal 4. What leaves a house is a reduced, signed card, never a measurement; what arrives is a hint to be validated locally. Dashed boxes are the Phase 2 design.

9.2 What is shared, what stays local, and why

Table 17. Sharing boundary.

Item	Shared?	Reason
Raw voltage, current and power series; per-plug data	no	They reveal the habits of the occupants
Identity, address, precise timestamp	no (time of day only)	Re-identification risk
Anomaly type, affected channels, severity and duration (in buckets)	yes	The useful knowledge, reduced
Numerical signature of the anomaly (standardised, rounded)	yes	Lets another house test the hypothesis on its own data; may reveal the source
Calibration statistics and sensor error models	yes	Lets a new house start well calibrated
Log, operator identities, thresholds, policy	no	Authority stays local; a card can only suggest
Model weights or gradients	no	Federated learning: Phase 2

9.3 From a verified case to shared knowledge

1. **Verified outcome.** A label is never produced by the detector itself: it comes from operator feedback and reviewed evidence.
2. **Candidate card.** The outcome is reduced to a minimal card: equipment class, signature, supported checks, result, scope and expiry.
3. **Validation.** Origin, label evidence, duplicates and sensitive fields are checked.
4. **Human review.** A reviewer approves the sharing scope.
5. **Indexing.** Only approved cards enter the shared library: lineage in Neo4j, search in Qdrant, with scope, validity and review status. A revocation blocks retrieval before the index entry is deleted, and the audit references are kept.
6. **Use by another house.** The card is a *diagnostic hint*: it never lowers a threshold and never triggers an action, and it is used only after a **staged gate** on the receiving house's own calibration data.

Model promotion is separate: a reproducible bundle (preprocessing, weights, feature schema, calibration), an evaluation gate, human approval, shadow validation before activation, no silent replacement, and a rollback plan. **Collective**

defence: a bias pattern newly seen at one house is proposed to a compatible house as a diagnostic candidate, never as an automatic corrective rule. **Federated learning** (a federated NILM direction [35]) is a later extension: houses are grouped by feature schema and equipment, negative transfer is tested, and low-quality or malicious updates are handled [36]. Federation alone does not guarantee privacy; secure aggregation and differential privacy are separate decisions with a cost in accuracy.

Table 18. Threat model for shared learning.

Threat	Safeguard
Poisoned card or model update	Reviewed cards only; signed origin; evaluation on held-out data; staged activation and rollback
Re-identification through signatures or embeddings	Data reduction; k-anonymity threshold (planned); pseudonyms alone are not enough
Forged signature or stolen key	HMAC signature [21] with enrolled keys; asymmetric signatures and key rotation (planned)
Negative transfer between different houses	Group by feature schema and equipment; measure the effect before enabling sharing

Phase 1 prototype. A two-house simulation implements signed cards (about 1.35 KB, 30-day expiry), the ordered checks and the staged gate. Its safety behaviour is as designed: forged signatures rejected (50 of 50), cards from another equipment type refused (0 of 82 accepted), poisoned cards accepted by the checks but not enabled because the gate stayed closed. However, sharing shows no measured gain, and the signatures are not anonymous (the source house is found with 0.73 accuracy against 0.167 by chance): these are limits that Phase 2 addresses.

10 Implementation plan and risks

Phase 1 (until 15 October 2026), delivered: data loaders; a seeded injector and a leakage audit; baseline models and detector v1 with calibration and abstention; rule-backed explanations with SHAP; a deny-by-default policy; a hash-chained log; an operator-interface prototype; the feeder, fusion and sharing experiments; a published dataset; tests and a public repository.

Phase 2 (until 12 December 2026), planned, ending with the full report, the repository with trained models, the demonstration video and the presentation. **Demonstration planned for Phase 2:** replay a real grid event, an isolated meter bias and a coordinated reporting inconsistency, showing for each the evidence, the explanation and the authorisation outcome; include a benign case, an ambiguous case and a denied unsafe request; share one validated lesson between two houses; finish with measured comparisons to baselines.

Table 19. Phase 2 roadmap, in priority order. Everything is done on recordings and simulation, as the brief expects.

	Item	Why
1	Inject anomalies into the published dataset and rerun detection there	The published dataset is still anomaly-free
2	Improve detection: per-house adaptation, richer physics checks, better recall on the fault types that are missed	Transfer fails; recall of three fault types collapses
3	Measure the improvement against baselines on the four criteria of the brief: precision and recall, false-positive rate, explanation quality, response time	Numbers the jury can compare
4	Explainability: add LIME next to SHAP	A second, model-independent opinion
5	Complete the prototype of the house node: thresholds, review interface, safeguards; service layer (ledger, graph, case index) and investigation workflow with a policy engine	Stand-ins are in place
6	Sharing as a documented design: harder benefit test, k-anonymity, asymmetric signatures	No benefit shown yet; signatures are not anonymous
7	Optional extensions (NILM, graph model), kept only if they beat simpler baselines	Avoid unnecessary complexity

Table 20. Main risks and mitigations.

Risk	Mitigation
Synthetic injections do not represent real faults; shortcuts inflate results	Leakage audit, held-out houses and a second dataset; evaluation on real recordings; field validation is a step beyond the challenge
Transfer between houses and datasets fails	Reported openly; per-house adaptation planned
Alarm fatigue; ambiguous cases	False alarms per day reported; unresolved cases routed to a human
Automation over-trust; unsafe action	Deny-by-default policy; human-only mutation; safeguards
Poisoned or identifying shared cards	Signed, reviewed, scoped cards; a gate that fails closed; privacy probe
Small team, short timeline	Core trust layer first; scope frozen before the evaluation

11 Business scope, feasibility and conclusion

Table 21. Business scope.

Item	Assumption	Cost driver
Implementation	Install next to the utility's meter-data system and reuse its feed	Integration engineer-weeks
Production	Containerised services, policy under change control, log on append-only storage	Operations, storage
Continuous training	Periodic retraining on new normal data, recalibration, drift monitoring, per-house adaptation	Data-engineer time, compute
Operator integration	Queue into existing ticketing or supervision tools; operator training	Training, interface work
Compliance	Data-protection review before any sharing between houses	Legal review

11.1 Business scope: budget, implementation, production, training

The prototype runs on a laptop-class machine with open-source tools; the real costs lie elsewhere. No price is given below because none was verified; prices must come from quotes. **Operations:** we would start on a single workstation with Docker Compose, track ingestion lag, missing data, job latency, unsupported checks and policy denials, back up the ledger, grid graph and artifacts, and rebuild the Qdrant index from approved content. **Deployment path:** shadow mode first (flags only, no actions); then reversible automatic actions once measured false-alarm rates are acceptable; humans only for anything that changes an asset. The prototype is not a certified protective controller.

11.2 Feasibility, sustainability and impact

Technical: open-source tools, millisecond inference, no special hardware. **Economic:** the software is inexpensive; the costs are integration and, where absent, instrumentation; no saving is asserted; it would have to be measured in a field trial after the challenge. **Regulatory:** low-voltage self-producers may sell surplus to STEG [26]; low-voltage connections stay within $\pm 10\,\%$ of nominal [27]; load curves reveal occupancy, so a legal review precedes any sharing; a model score is not proof of theft, so suspected falsification always goes to a human. **Sustainable Development Goals:** SDG 7 (trustworthy data to integrate rooftop solar and reduce the peak), SDG 9 (a transparent, auditable digital layer on grid infrastructure), SDG 11 (more reliable supply, fewer unnecessary interventions), SDG 13 (less reliance on gas at the peak).

11.3 Limits and conclusion

Every detection number measures how well we find our own fault generators on real house recordings; there are no real labelled faults, no field deployment and no operator study. The operating point trades recall for precision, unseen houses lose about 0.2 F1, transfer between datasets fails, sharing shows no measured benefit, the published dataset does not contain injected anomalies yet, and its consumption levels are above the Tunisian average.

Phase 1 delivers a trust layer that separates measurement problems from real grid events, says how sure it is and why, routes decisions through a policy it cannot override, and defines a safe protocol for sharing between houses, together with an open dataset and a benchmark whose weaknesses we found and fixed ourselves. Phase 2 improves detection, injects anomalies into the published dataset and completes the explanation and governance layers. Validation on real meter data and with real operators is the natural next step beyond the challenge.

A Repository and reproducibility

Code: `https://github.com/azert88/energymind-tunisia`
Dataset (Kaggle): `https://www.kaggle.com/datasets/azertyqsdfghqsdfgh/pes-test`
Splits are contiguous time blocks and injection is seeded. Raw third-party recordings are not in the repository (licence and size); the data notes explain how to obtain and map them.

Table 22. Repository map.

Path	Purpose
`energymind/data_*.py`, `inject.py`	Canonical frames; seeded injector and splits
`energymind/features*.py`, `models_*.py`, `evaluate.py`	Features, models, event metrics, calibration, abstention
`energymind/explain.py`, `policy.py`, `decision_log.py`	SHAP and templates; deny-by-default routing; hash-chained log
`energymind/feeder.py`, `fusion.py`, `sharing.py`	Feeder diagnostic, fusion, case cards
`app/operator.py`, `configs/policy.yaml`	Operator interface; policy file
`scripts/`, `tests/`, `results/`, `docs/`, `data/DATA_NOTES*.md`	Runners; unit tests; measured outputs; design and data notes

Main commands:

```
python -m venv .venv ; .venv\Scripts\pip install -r requirements.txt
python scripts\build_data.py              # canonical 1-minute frames
python -m pytest tests                    # unit tests
python scripts\run_experiments.py         # baseline models, injector v2
python scripts\run_deep.py --seeds 3      # TCN and autoencoder
python scripts\leakage_audit.py           # shortcut audit, legacy versus v2
python scripts\run_improve.py             # detector v1 pipeline
python scripts\demo_end_to_end.py         # operator demo on injected faults
streamlit run app\operator.py             # operator interface
```

References

[1] Maghreb Émergent (6 Oct 2026). *Électricité : la Tunisie cherche à renforcer sa production après un été sous tension* (citing the National Energy Observatory). https://maghrebemergent.news/fr/electricite-la-tunisie-cherche-a-renforcer-sa-production-apres-un-ete-sous-tension/

[2] Webdo (15 Jul 2026). *Tunisie : pourquoi la STEG multiplie les coupures d'électricité*. https://www.webdo.tn/fr/actualite/national/tunisie-pourquoi-la-steg-multiplie-les-coupures-delectricite/401071/

[3] L'Économiste Maghrébin (31 Aug 2026). *Faut-il détester le délestage électrique ?* https://www.leconomistemaghrebin.com/2026/08/31/faut-il-detester-le-delestage-electrique-ce-que-revele-la-crise-tunisienne-de-lete-2026/

[4] Tunisie Numérique (Oct 2026). *Électricité en Tunisie : les 11 mesures de la STEG*. https://www.tunisienumerique.com/electricite-tunisie-11-mesures-steg-2026/

[5] Webdo (2026). *Énergie solaire : plus de 150.000 installations et 420 MW raccordés au réseau basse tension* (citing ANME). https://www.webdo.tn/fr/actualite/national/energie-solaire-plus-de-150-000-installations-et-420-mw-raccordes-au-reseau-basse-tension/403753/

[6] IEEE TSYP14 (2026). EnergyMind Tunisia Technical Challenge, challenge brief.

[7] Beckel C. et al. (2014). The ECO data set and the performance of non-intrusive load monitoring algorithms. *BuildSys*. https://vs.inf.ethz.ch/res/show.html?what=eco-data

[8] Makonin S. et al. (2016). Electricity, water, and natural gas consumption of a residential house in Canada from 2012 to 2014 (AMPds2). *Scientific Data*.

[9] Schlemminger M. et al. (2022). Dataset on electrical single-family house and heat pump load profiles in Germany. *Scientific Data*. https://zenodo.org/records/5642902

[10] Meinecke S. et al. (2020). SimBench: a benchmark dataset of electric power systems to compare innovative solutions based on power flow analysis. *Energies* 13(12), 3290. https://doi.org/10.3390/en13123290 Licence: https://simbench.readthedocs.io/en/stable/about/license.html

[11] Thurner L. et al. (2018). pandapower, an open-source Python tool for modeling, analysis and optimization of electric power systems. *IEEE Trans. Power Systems* 33(6), 6510–6521. https://doi.org/10.1109/TPWRS.2018.2829021

[12] European Commission Joint Research Centre. PVGIS. https://re.jrc.ec.europa.eu/pvg_tools/en/

[13] EN 50160. Voltage characteristics of electricity supplied by public electricity networks (CENELEC).

[14] Open Data Commons. Open Database License (ODbL) 1.0. https://opendatacommons.org/licenses/odbl/1.0/

[15] Ke G. et al. (2017). LightGBM: a highly efficient gradient boosting decision tree. *NeurIPS* 30, 3146–3154. https://papers.nips.cc/paper/6907-lightgbm-a-highly-efficient-gradient-boosting-decision-tree

[16] Guo C. et al. (2017). On calibration of modern neural networks. *ICML*, PMLR 70:1321–1330. https://proceedings.mlr.press/v70/guo17a.html

[17] Tatbul N. et al. (2018). Precision and recall for time series. *NeurIPS* 31, https://proceedings.neurips.cc/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html; Kim S. et al. (2022). Towards a rigorous evaluation of time-series anomaly detection. *AAAI* 36(7), 7194–7201. https://doi.org/10.1609/aaai.v36i7.20680

[18] Lundberg S. et al. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence* 2(1), 56–67. https://doi.org/10.1038/s42256-019-0138-9

[19] Ribeiro M. et al. (2016). "Why should I trust you?" Explaining the predictions of any classifier (LIME). *KDD*. https://arxiv.org/abs/1602.04938

[20] Crosby S., Wallach D. (2009). Efficient data structures for tamper-evident logging. *USENIX Security Symposium*. https://www.usenix.org/conference/usenixsecurity09/technical-sessions/presentation/efficient-data-structures-tamper-evident

[21] Krawczyk H., Bellare M., Canetti R. (1997). HMAC: keyed-hashing for message authentication. RFC 2104.

[22] Zhang C. et al. (2018). Sequence-to-point learning with neural networks for non-intrusive load monitoring. *AAAI*. https://ojs.aaai.org/index.php/AAAI/article/view/11873

[23] Neo4j GraphRAG documentation. https://neo4j.com/docs/neo4j-graphrag-python/current/

[24] Qdrant documentation. https://qdrant.tech/documentation/

[25] African Manager. *Les ménages tunisiens utilisent de plus en plus les lampes économiques* (citing a STEG survey of 2,850 low-voltage customers; average household electricity consumption about 1,636 in 2019, unit written "kw" in the article and read here as kWh per year). https://africanmanager.com/les-menages-tunisiens-utilisent-de-plus-en-plus-les-lampes-economiques/

[26] République tunisienne. Décret gouvernemental n° 2016-1123 (production d'électricité à partir d'énergies renouvelables). https://rise.esmap.org/data/files/library/tunisia/Renewable%20Energy/Tunisia_Decree%20N%C2%B02016-1123%20Renewable%20energy%20-%20Fr.pdf

[27] République tunisienne. Décision du 9 février 2017, code de réseau basse tension. https://rise.esmap.org/data/files/library/tunisia/Energy%20Efficiency/Tunisia_Decision%20of%2009%20F%C3%A9vrier%202017%20-%20Grid%20Code%20LV%20-%20%20FR.pdf

[28] Gibbs I., Candès E. (2024). Conformal inference for online prediction with arbitrary distribution shifts. *JMLR* 25. https://jmlr.org/papers/v25/22-1218.html

[29] Sundararajan M. et al. (2017). Axiomatic attribution for deep networks. https://arxiv.org/abs/1703.01365

[30] Ying R. et al. (2019). GNNExplainer: generating explanations for graph neural networks. https://arxiv.org/abs/1903.03894

[31] LangGraph: interrupts and human-in-the-loop workflows. https://docs.langchain.com/oss/python/langgraph/interrupts

[32] Open Policy Agent documentation. https://www.openpolicyagent.org/docs/latest/

[33] Multilingual E5 small model card. https://huggingface.co/intfloat/multilingual-e5-small

[34] Qwen2.5-7B-Instruct model card. https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[35] A federated time pattern-based framework for privacy-preserving distributed non-intrusive load monitoring (2025). https://www.mdpi.com/2075-1702/13/8/718

[36] McMahan B. et al. (2017). Communication-efficient learning of deep networks from decentralized data. *AISTATS*, PMLR 54:1273–1282. https://proceedings.mlr.press/v54/mcmahan17a.html